

Recorridos de grafos

Programación III - UNGS

Recorrido de un grafo

- **Problema.** Determinar si un grafo es conexo.

Recorrido de un grafo

- **Problema.** Determinar si un grafo es conexo.
- **Idea.** Partir de un vértice $s \in V$ arbitrario, y obtener todos los vértices que se pueden **alcanzar** a partir de s .

Recorrido de un grafo

- **Problema.** Determinar si un grafo es conexo.
- **Idea.** Partir de un vértice $s \in V$ arbitrario, y obtener todos los vértices que se pueden **alcanzar** a partir de s .
 1. $L := \{s\};$

Recorrido de un grafo

- **Problema.** Determinar si un grafo es conexo.
- **Idea.** Partir de un vértice $s \in V$ arbitrario, y obtener todos los vértices que se pueden **alcanzar** a partir de s .
 1. $L := \{s\};$
 2. Mientras $L \neq \emptyset$:

Recorrido de un grafo

- **Problema.** Determinar si un grafo es conexo.
- **Idea.** Partir de un vértice $s \in V$ arbitrario, y obtener todos los vértices que se pueden **alcanzar** a partir de s .
 1. $L := \{s\};$
 2. Mientras $L \neq \emptyset$:
 3. Seleccionar $i \in L$ y marcarlo;

Recorrido de un grafo

- **Problema.** Determinar si un grafo es conexo.
- **Idea.** Partir de un vértice $s \in V$ arbitrario, y obtener todos los vértices que se pueden **alcanzar** a partir de s .
 1. $L := \{s\};$
 2. Mientras $L \neq \emptyset$:
 3. Seleccionar $i \in L$ y marcarlo;
 4. Agregar a L todos los vecinos no marcados de i ;

Recorrido de un grafo

- **Problema.** Determinar si un grafo es conexo.
- **Idea.** Partir de un vértice $s \in V$ arbitrario, y obtener todos los vértices que se pueden **alcanzar** a partir de s .
 1. $L := \{s\};$
 2. Mientras $L \neq \emptyset$:
 3. Seleccionar $i \in L$ y marcarlo;
 4. Agregar a L todos los vecinos no marcados de i ;
 5. $L := L \setminus \{i\};$

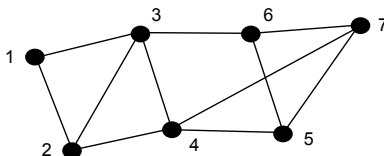
Recorrido de un grafo

- **Problema.** Determinar si un grafo es conexo.
- **Idea.** Partir de un vértice $s \in V$ arbitrario, y obtener todos los vértices que se pueden **alcanzar** a partir de s .
 1. $L := \{s\};$
 2. Mientras $L \neq \emptyset$:
 3. Seleccionar $i \in L$ y marcarlo;
 4. Agregar a L todos los vecinos no marcados de i ;
 5. $L := L \setminus \{i\};$
 6. Fin

Recorrido de un grafo

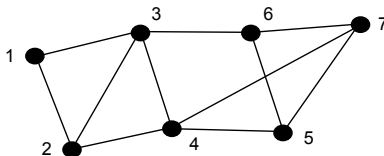
- **Problema.** Determinar si un grafo es conexo.
- **Idea.** Partir de un vértice $s \in V$ arbitrario, y obtener todos los vértices que se pueden **alcanzar** a partir de s .
 1. $L := \{s\}$;
 2. Mientras $L \neq \emptyset$:
 3. Seleccionar $i \in L$ y marcarlo;
 4. Agregar a L todos los vecinos no marcados de i ;
 5. $L := L \setminus \{i\}$;
 6. Fin
 7. Los vértices marcados son los alcanzables desde s ;

Recorrido de un grafo



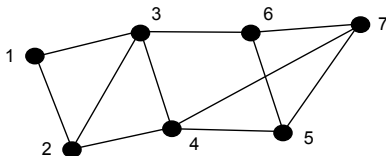
- Si la lista L se implementa con una **cola**, entonces el algoritmo se llama BFS (*breadth-first search*), y recorre los vértices en orden de distancia creciente desde s .

Recorrido de un grafo



- Si la lista L se implementa con una **cola**, entonces el algoritmo se llama BFS (*breadth-first search*), y recorre los vértices en orden de distancia creciente desde s .
- Si la lista L se implementa con una **pila**, entonces el algoritmo se llama DFS (*depth-first search*), y encuentra primero el vértice más lejano a s .

Recorrido de un grafo



- Si la lista L se implementa con una **cola**, entonces el algoritmo se llama BFS (*breadth-first search*), y recorre los vértices en orden de distancia creciente desde s .
- Si la lista L se implementa con una **pila**, entonces el algoritmo se llama DFS (*depth-first search*), y encuentra primero el vértice más lejano a s .
- Si los vértices marcados son todos los vértices, el grafo es conexo.

Recorrido de un grafo

- ¿Qué **complejidad computacional** tiene este algoritmo?
1. $O(1)$
 2. Mientras $L \neq \emptyset$:
 3. Para todos los vecinos no marcados de i :
 4. $O(1)$
 5. $O(1)$
 6. Fin

Recorrido de un grafo

- ¿Qué **complejidad computacional** tiene este algoritmo?
 1. $O(1)$
 2. Mientras $L \neq \emptyset$:
 3. Para todos los vecinos no marcados de i :
 4. $O(1)$
 5. $O(1)$
 6. Fin
- El ciclo exterior realiza $n := |V|$ iteraciones en el peor caso.

Recorrido de un grafo

- ¿Qué **complejidad computacional** tiene este algoritmo?
 1. $O(1)$
 2. Mientras $L \neq \emptyset$:
 3. Para todos los vecinos no marcados de i :
 4. $O(1)$
 5. $O(1)$
 6. Fin
- El ciclo exterior realiza $n := |V|$ iteraciones en el peor caso.
- ¿Cuál es la complejidad computacional de obtener todos los vecinos no marcados?

Recorrido de un grafo

- ¿Qué **complejidad computacional** tiene este algoritmo?
 1. $O(1)$
 2. Mientras $L \neq \emptyset$:
 3. Para todos los vecinos no marcados de i :
 4. $O(1)$
 5. $O(1)$
 6. Fin
- El ciclo exterior realiza $n := |V|$ iteraciones en el peor caso.
- ¿Cuál es la complejidad computacional de obtener todos los vecinos no marcados?
 1. Si el grafo está implementado sobre una matriz de adyacencia, obtener los vecinos es $O(n)$.

Recorrido de un grafo

- ¿Qué **complejidad computacional** tiene este algoritmo?
 1. $O(1)$
 2. Mientras $L \neq \emptyset$:
 3. Para todos los vecinos no marcados de i :
 4. $O(1)$
 5. $O(1)$
 6. Fin
- El ciclo exterior realiza $n := |V|$ iteraciones en el peor caso.
- ¿Cuál es la complejidad computacional de obtener todos los vecinos no marcados?
 1. Si el grafo está implementado sobre una matriz de adyacencia, obtener los vecinos es $O(n)$.
 2. En este caso, el algoritmo completo es $O(n^2)$.

Recorrido de un grafo

- Sin embargo, si el grafo está implementado sobre listas de vecinos:
 1. Obtener los vecinos de un vértice es $O(1)$.

Recorrido de un grafo

- Sin embargo, si el grafo está implementado sobre listas de vecinos:
 1. Obtener los vecinos de un vértice es $O(1)$.
 2. Recorrer la lista de vecinos es $O(d(i))$.

Recorrido de un grafo

- Sin embargo, si el grafo está implementado sobre listas de vecinos:
 1. Obtener los vecinos de un vértice es $O(1)$.
 2. Recorrer la lista de vecinos es $O(d(i))$.
- Podemos realizar el siguiente análisis
 1. $O(1)$
 2. Mientras $L \neq \emptyset$:
 3. Procesar los vecinos de i : $O(d(i))$
 4. $O(1)$
 5. Fin

Recorrido de un grafo

- Sin embargo, si el grafo está implementado sobre listas de vecinos:
 1. Obtener los vecinos de un vértice es $O(1)$.
 2. Recorrer la lista de vecinos es $O(d(i))$.
- Podemos realizar el siguiente análisis
 1. $O(1)$
 2. Mientras $L \neq \emptyset$:
 3. Procesar los vecinos de i : $O(d(i))$
 4. $O(1)$
 5. Fin
- Entonces, en este caso la complejidad computacional del algoritmo es $\sum_{i \in V} O(d(i))$.

Recorrido de un grafo

- Llamamos $m := |E|$ a la cantidad de aristas del grafo.

Recorrido de un grafo

- Llamamos $m := |E|$ a la cantidad de aristas del grafo.
- **Handshaking Lemma.** $\sum_{i \in V} d(i) = 2m$.

Recorrido de un grafo

- Llamamos $m := |E|$ a la cantidad de aristas del grafo.
- **Handshaking Lemma.** $\sum_{i \in V} d(i) = 2m$.
- Entonces, la complejidad computacional de BFS es:
 1. $O(n^2)$ si el grafo está implementado sobre una matriz de adyacencia.
 2. $O(m)$ si el grafo está implementado sobre listas de vecinos.

Recorrido de un grafo

- Llamamos $m := |E|$ a la cantidad de aristas del grafo.
- **Handshaking Lemma.** $\sum_{i \in V} d(i) = 2m$.
- Entonces, la complejidad computacional de BFS es:
 1. $O(n^2)$ si el grafo está implementado sobre una matriz de adyacencia.
 2. $O(m)$ si el grafo está implementado sobre listas de vecinos.
- Es habitual escribir la complejidad de algoritmos sobre grafos en función de $n = |V|$ y $m = |E|$.

Recorrido de un grafo

- Llamamos $m := |E|$ a la cantidad de aristas del grafo.
- **Handshaking Lemma.** $\sum_{i \in V} d(i) = 2m$.
- Entonces, la complejidad computacional de BFS es:
 1. $O(n^2)$ si el grafo está implementado sobre una matriz de adyacencia.
 2. $O(m)$ si el grafo está implementado sobre listas de vecinos.
- Es habitual escribir la complejidad de algoritmos sobre grafos en función de $n = |V|$ y $m = |E|$.
 1. En general, $m = O(n^2)$, con lo cual el peor caso es el mismo para ambas implementaciones.

Recorrido de un grafo

- Llamamos $m := |E|$ a la cantidad de aristas del grafo.
- **Handshaking Lemma.** $\sum_{i \in V} d(i) = 2m$.
- Entonces, la complejidad computacional de BFS es:
 1. $O(n^2)$ si el grafo está implementado sobre una matriz de adyacencia.
 2. $O(m)$ si el grafo está implementado sobre listas de vecinos.
- Es habitual escribir la complejidad de algoritmos sobre grafos en función de $n = |V|$ y $m = |E|$.
 1. En general, $m = O(n^2)$, con lo cual el peor caso es el mismo para ambas implementaciones.
 2. Si $m = O(n)$, decimos que el grafo es **poco denso** y entonces conviene la segunda implementación.